BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Tô Gia Huy
MSSV / Lớp	DCT1232

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

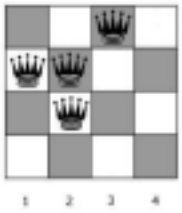
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

bàn cờ có 4 quân hậu, mỗi cột chứa đúng 1 quân hậu. Trạng thái được biểu diễn bằng bộ 4 số (x_1, x_2, x_3, x_4), trong đó x_i là hàng đặt quân hậu ở cột i.

Quan sát hình, ta có:

- Cột 1: hậu ở hàng 2.
- Cột 2: hậu ở hàng 3.
- Cột 3: hậu ở hàng 4.
- Cột 4: hậu ở hàng 1.

Vậy trạng thái khởi đầu là:

$S_0 = (2, 3, 4, 1)$ ✅

Mỗi vòng lặp, thuật toán di chuyển một quân hậu trong cùng cột sang hàng khác, sao cho số cặp hậu tấn công nhau giảm xuống thấp nhất có thể.

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

✅ A) Không dừng

✅ B) Cực trị cục bộ (local optimum)

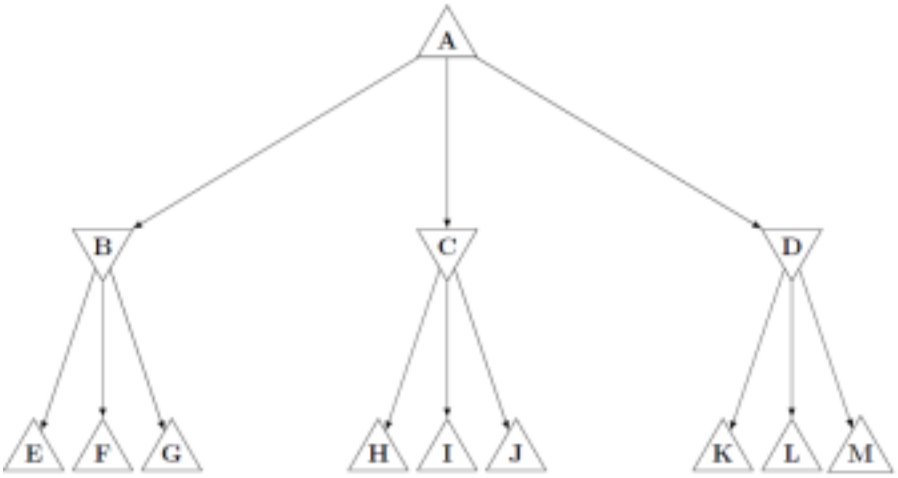
☐ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

✅ B → C → D

☐ C → D → B

☐ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Áp dụng Alpha-Beta theo thứ tự $D \rightarrow C \rightarrow B$, giữ nguyên thứ tự các nút lá từ trái sang phải.

Bước 1: Xét cây D

- Duyệt $K = 23,\ L = 20,\ M = 26$.
- Giá trị nút $D = \min(23, 20, 26) = 20$.
- Cập nhật $\alpha = 20$ tại nút A.

Bước 2: Xét cây C

- Duyệt $H = 17$, cập nhật $\beta = 17$ tại C.
- Vì $\beta \leq \alpha$, tức $17 \leq 20$, cắt tỉa các nút còn lại I, J.

Bước 3: Xét cây B

- Duyệt $E = 5$, cập nhật $\beta = 5$ tại B.
- Vì $\beta \leq \alpha$, tức $5 \leq 20$, cắt tỉa các nút còn lại F, G.

Vậy các nút không được thăm là: I,J,F,G

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

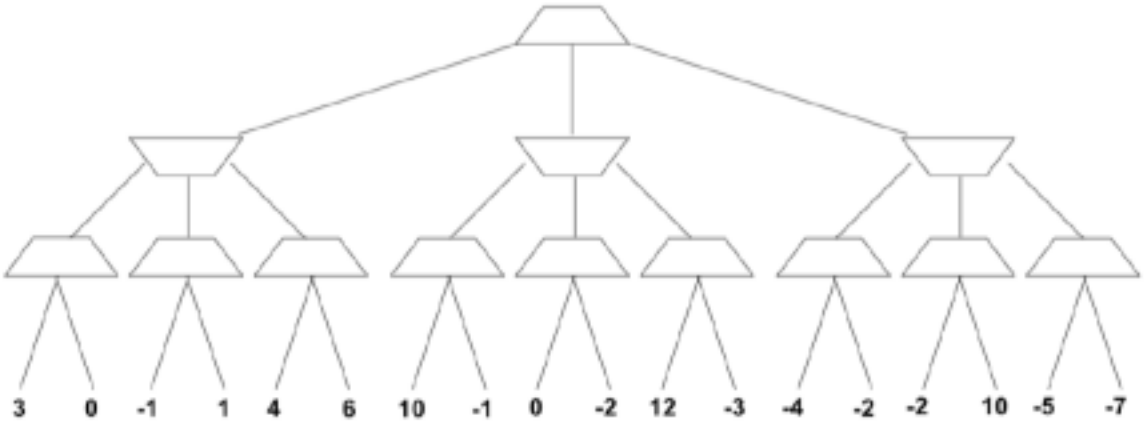
✅ Có

☐ Không

Alpha-Beta Pruning chỉ loại bỏ những nhánh không thể ảnh hưởng đến kết quả Minimax. Iterative Deepening tìm kiếm tăng dần theo độ sâu và không làm thay đổi tính đúng đắn của Alpha-Beta.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

1. Tính giá trị Minimax

Các nút MAX ở tầng dưới cùng có hai lá con mỗi nút:

Nút MAX	Giá trị lá	Giá trị trả về
A_1	$3, 0$	3
A_2	$-1, 1$	1
A_3	$4, 6$	6
A_4	$2, -3$	2
A_5	$-2, -3$	-2
A_6	$4, 3$	4
A_7	$-2, -2$	-2

A_8	$10, -5$	10
A_9	$-5, -7$	-5

Các nút MIN ở tầng giữa có giá trị:

$$B_1 = \min(3, 1, 6) = 1$$
$$B_2 = \min(2, -2, 4) = -2$$
$$B_3 = \min(-2, 10, -5) = -5$$

Vì vậy, giá trị tại nút gốc MAX là:

V(A)=max(1,-2,-5)=1

2. Duyệt Alpha-Beta và xác định nhánh bị cắt

Ban đầu tại gốc MAX:

$$\alpha = -\infty, \beta = +\infty$$

Nhánh trái B_1

- $A_1 = (3, 0) \Rightarrow 3$, nên $\beta_{B_1} = 3$.
- $A_2 = (-1, 1) \Rightarrow 1$, nên $\beta_{B_1} = 1$.
- $A_3 = (4, 6)$: kế thừa $\beta = 1$. Xét lá 4, ta có $\alpha = 4 \geq \beta = 1$, nên cắt lá 6.
- $B_1 = 1$. Cập nhật tại gốc: $\alpha = 1$.

Nhánh giữa B_2

- $A_4 = (2, -3) \Rightarrow 2$, nên $\beta_{B_2} = 2$.
- $A_5 = (-2, -3) \Rightarrow -2$, nên $\beta_{B_2} = -2$.
- Vì $\beta = -2 \leq \alpha = 1$, cắt toàn bộ cây con A_6, gồm hai lá 4, 3.
- $B_2 = -2$.

Nhánh phải B_3

- $A_7 = (-2, -2) \Rightarrow -2$, nên $\beta_{B_3} = -2$.

- Vì $\beta = -2 \leq \alpha = 1$, cắt toàn bộ hai cây con A_8, A_9, gồm các lá $10, -5, -5, -7$.

3. Tổng hợp kết quả

Nút	Giá trị trả về / kết luận	α	β
Gốc MAX	1	1	$+\infty$
B_1 MIN	1	$-\infty$	1
B_2 MIN	-2	1	-2
B_3 MIN	Không cần tính chính xác; giá trị ≤ -2	1	-2

Các giá trị α, β ở bảng là giá trị tiêu biểu tại thời điểm kết thúc xử lý từng nhánh; trong quá trình duyệt, chúng được cập nhật liên tục.

Các nhánh bị cắt tỉa:

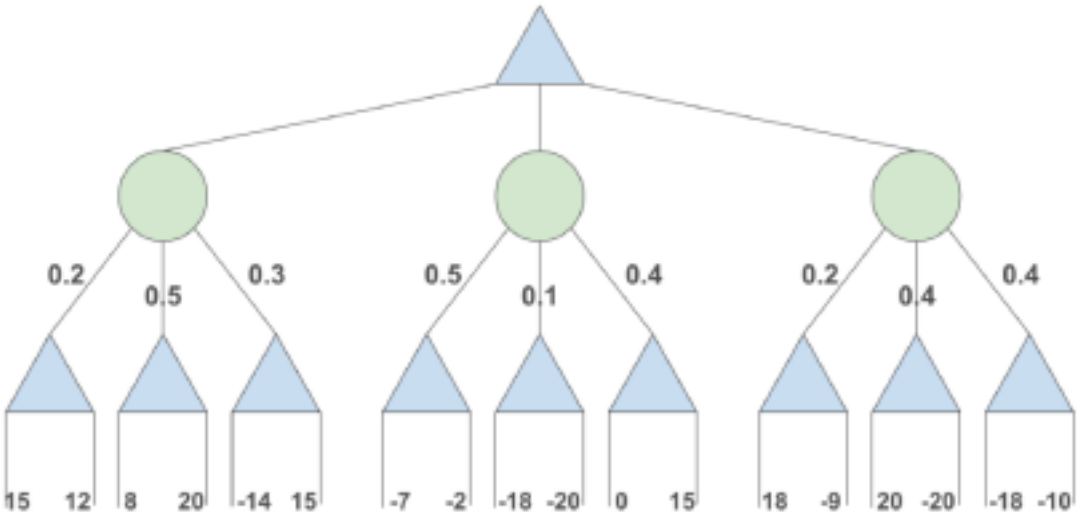
- Lá 6 thuộc A_3.
- Hai lá 4, 3 thuộc A_6.
- Hai cây con A_8, A_9, gồm các lá $10, -5, -5, -7$.

4. Kết luận

- Giá trị Minimax tại gốc:1.
- Nước đi tối ưu của MAX: chọn nhánh trái B_1.
- Alpha-Beta Pruning giúp giảm số nút cần duyệt nhưng vẫn giữ nguyên kết quả Minimax.

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Nút MAX: chọn giá trị lớn nhất trong các nút con.

$MAX = \max(v_1, v_2, ..., v_□)$

- Nút Chance: tính kỳ vọng theo xác suất của các nhánh.

$E = p_1v_1 + p_2v_2 + ... + p_□v_□$

Bước 1: Tính giá trị các nút MAX ở tầng dưới cùng

Nhánh trái:

- $MAX_1 = \max(15, 12) = 15$
- $MAX_2 = \max(8, 20) = 20$
- $MAX_3 = \max(-14, 15) = 15$

Nhánh giữa:

- $MAX_4 = \max(-7, -2) = -2$
- $MAX_5 = \max(-18, -20) = -18$
- $MAX_6 = \max(0, 15) = 15$

Nhánh phải:

- $MAX_7 = \max(18, -9) = 18$
- $MAX_8 = \max(20, -20) = 20$
- $MAX_9 = \max(-18, -10) = -10$

Bước 2: Tính giá trị các Chance Node

Chance Node trái:

$E_1 = 0.2 \times 15 + 0.5 \times 20 + 0.3 \times 15$

$E_1 = 3 + 10 + 4.5 = 17.5$

Chance Node giữa:

$E_2 = 0.5 \times (-2) + 0.1 \times (-18) + 0.4 \times 15$

$E_2 = -1 - 1.8 + 6 = 3.2$

Chance Node phải:

$E_3 = 0.2 \times 18 + 0.4 \times 20 + 0.4 \times (-10)$

$E_3 = 3.6 + 8 - 4 = 7.6$

Bước 3: Tính giá trị nút gốc MAX

MAX_gốc = max(17.5, 3.2, 7.6)

MAX_gốc = 17.5

Vậy nút gốc chọn nhánh trái.

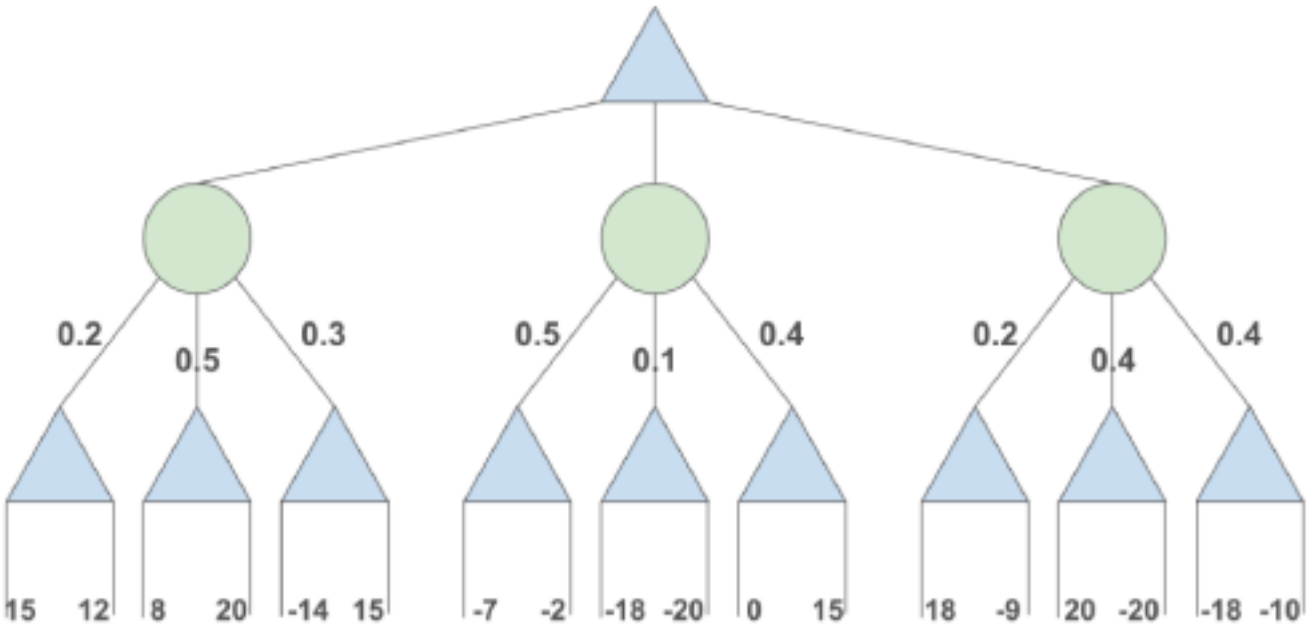
b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Đáp án: Không thể cắt tỉa chắc chắn nút nào chỉ dựa vào các thông tin đã cho.

Giải thích:

- Với Expectimax, giá trị của Chance Node phụ thuộc vào giá trị của tất cả các nhánh và xác suất tương ứng.
- Khi chưa biết giới hạn trên hoặc giới hạn dưới của các giá trị còn chưa xét, ta không thể kết luận rằng một nhánh chưa duyệt sẽ không làm thay đổi giá trị kỳ vọng.
- Vì vậy, trong trường hợp tổng quát, cần duyệt các nhánh để tính chính xác giá trị kỳ vọng.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Ta duyệt từ trái sang phải.

Bước 1: Duyệt Chance Node trái

Giá trị chính xác của nhánh trái là:

$E_1 = 17.5$

Do đó, giá trị tốt nhất hiện tại tại nút gốc MAX là:

$\alpha = 17.5$

Bước 2: Duyệt Chance Node giữa

Giá trị MAX đầu tiên:

$MAX_4 = \max(-7, -2) = -2$

Giá trị MAX thứ hai:

$MAX_5 = \max(-18, -20) = -18$

Giá trị MAX thứ ba chưa xét, nhưng vì mọi nút lá nằm trong [-20, 20], nên giá trị của MAX_6 không thể vượt quá 20.

Giá trị kỳ vọng lớn nhất có thể đạt được của Chance Node giữa là:

$E_2_max = 0.5 \times (-2) + 0.1 \times (-18) + 0.4 \times 20$

$E_2_max = -1 - 1.8 + 8 = 5.2$

Vì 5.2 < 17.5, có thể kết luận nhánh giữa không thể tốt hơn nhánh trái. Tuy nhiên, để đưa ra kết luận này, ta vẫn cần tính chính xác MAX_4 và MAX_5 như trên; không cần duyệt MAX_6, tức có thể cắt tỉa hai lá 0 và 15.

Bước 3: Duyệt Chance Node phải

Giá trị MAX đầu tiên:

$MAX_7 = \max(18, -9) = 18$

Giá trị MAX thứ hai:

$MAX_8 = \max(20, -20) = 20$

Giá trị MAX thứ ba chưa xét, nhưng có giá trị tối đa là 20.

Giá trị kỳ vọng lớn nhất có thể đạt được của Chance Node phải là:

$E_3_max = 0.2 \times 18 + 0.4 \times 20 + 0.4 \times 20$

$E_3_max = 3.6 + 8 + 8 = 19.6$

Vì 19.6 > 17.5, chưa thể cắt tỉa nhánh phải. Ta phải duyệt MAX_9:

$MAX_9 = \max(-18, -10) = -10$

Khi đó:

$E_3 = 0.2 \times 18 + 0.4 \times 20 + 0.4 \times (-10) = 7.6$

Kết quả cuối cùng:

- Chance Node trái = 17.5
- Chance Node giữa = 3.2
- Chance Node phải = 7.6
- Nút gốc MAX = 17.5

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Forward Checking: Sau khi gán giá trị cho một biến, thuật toán loại bỏ các giá trị không hợp lệ trong miền của các biến chưa gán có ràng buộc trực tiếp với biến vừa gán. Thuật toán không tiếp tục lan truyền sang các biến khác.

AC-3 (Arc Consistency Algorithm): Kiểm tra tính nhất quán của các cung trong CSP. Khi miền của một biến thay đổi, thuật toán có thể tiếp tục kiểm tra các cung liên quan để lan truyền ràng buộc đến nhiều biến khác.

So sánh:

- Phạm vi lan truyền: Forward Checking chỉ xét các biến chưa gán có liên quan trực tiếp; AC-3 có thể lan truyền qua nhiều cung.
- Thời điểm thực hiện: Forward Checking thường được thực hiện sau mỗi lần gán biến trong quá trình tìm kiếm; AC-3 có thể chạy trước khi tìm kiếm hoặc sau khi miền biến thay đổi.
- Chi phí tính toán: Forward Checking thường có chi phí thấp hơn nhưng phát hiện ít mâu thuẫn hơn; AC-3 thường tốn nhiều phép kiểm tra hơn nhưng có thể phát hiện mâu thuẫn sớm hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV chọn biến chưa gán có số lượng giá trị hợp lệ còn lại ít nhất trong miền.

Heuristic này giúp phát hiện mâu thuẫn sớm, vì biến có ít lựa chọn nhất dễ rơi vào tình trạng không còn giá trị hợp lệ. Nhờ đó, thuật toán tránh mất thời gian tìm kiếm sâu vào những nhánh không có lời giải.

ii) Least Constraining Value (LCV)

LCV chọn giá trị gây ít hạn chế nhất đối với miền giá trị của các biến chưa gán có liên quan.

Heuristic này giúp duy trì nhiều lựa chọn cho các biến còn lại, giảm nguy cơ dẫn đến mâu thuẫn và tăng khả năng tìm được lời giải mà không phải quay lui.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Một bài toán CSP gồm ba thành phần: biến X, miền giá trị D và tập ràng buộc C.

1. Tập biến

$X = \{A, B, C, D, E\}$

Mỗi biến đại diện cho một máy bay, cần xác định đường băng và khung giờ cất cánh.

2. Miền giá trị

Mỗi máy bay có hai thuộc tính:

- Đường băng: quốc tế (QT) hoặc nội địa (ND).
- Khung giờ: $1, 2, 3, 4$.

Có thể biểu diễn một giá trị là một cặp (r, t), trong đó $r \in \{QT, ND\}$ và $t \in \{1, 2, 3, 4\}$.

Do đó, miền ban đầu của mỗi biến là:

$$D_A = D_B = D_C = D_D = D_E = \{QT, ND\} \times \{1, 2, 3, 4\}$$

3. Các ràng buộc

- Máy bay B phải dùng động cơ và phải cất cánh ở khung giờ 1: $t_B = 1$.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi: $t_D \geq 3$.
- Máy bay A sẵn sàng nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2: $t_A \leq 2$.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh, vì có hành khách nối chuyến từ D sang C: $t_D < t_C$.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng:

$$\forall i \neq j, (r_i = r_j) \Rightarrow (t_i \neq t_j)$$

Sửa lại ràng buộc của máy bay B: Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1, nên ràng buộc đúng là $t_B = 1$.

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Do A, B, C chỉ sử dụng đường băng quốc tế; D, E chỉ sử dụng đường băng nội địa, ta chỉ cần xét biến khung giờ.

1. Miền giá trị

Sau khi áp dụng hai ràng buộc bổ sung về đường băng:

- A: 4

- B: 4
- C: 4
- D: 4
- E: 4

2. Đồ thị ràng buộc

- A, B, C dùng chung đường băng quốc tế nên mỗi cặp phải khác khung giờ: $A \neq B$, $A \neq C$, $B \neq C$.
- D, E dùng chung đường băng nội địa nên $D \neq E$.
- D phải hạ cánh trước C cất cánh: $D < C$.
- Các ràng buộc đơn: $B = 1$, $D \geq 3$, $A \leq 2$.

Đồ thị có các cạnh: A—B, A—C, B—C, D—E, D—C.

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Áp dụng các ràng buộc đơn trước:

- $B = 1 \Rightarrow$ miền $B = 1$.
- $D \geq 3 \Rightarrow$ miền $D = 4$.
- $A \leq 2 \Rightarrow$ miền $A = 2$.

Sau đó lan truyền tính nhất quán cung:

Bước 1: Vì A và B dùng chung đường băng, $A \neq B$. $B = 1$ nên loại 1 khỏi miền A.

$A = 2$.

Bước 2: A, B, C dùng chung đường băng.

- $A = 2 \Rightarrow$ loại 2 khỏi miền C.
- $B = 1 \Rightarrow$ loại 1 khỏi miền C.

$C = 4$.

Bước 3: Xét ràng buộc $D < C$.

- $D = 4$ không có giá trị C nào lớn hơn 4, nên loại 4 khỏi miền D.
- $D = 3$, vì vậy C phải lớn hơn 3, suy ra $C = 4$.

Kết quả: $D = 3$, $C = 4$.

Bước 4: D và E dùng chung đường băng, $D \neq E$.

$D = 3$ nên loại 3 khỏi miền E.

Miền giá trị cuối cùng:

- A = 2
- B = 1
- C = 4
- D = 3
- E = 4

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Áp dụng các ràng buộc đơn trước:

- A = 2
- B = 1
- C = 4
- D = 4
- E = 4

Bước 1: Chọn biến MRV

B có miền nhỏ nhất, B = 1.

Gán B = 1.

Forward Checking loại 1 khỏi miền A và C:

- A = 2

- C = 4

Bước 2: Chọn biến A

A có miền nhỏ nhất tiếp theo.

Gán A = 2.

Forward Checking loại 2 khỏi miền C:

- C = 4

Bước 3: Chọn biến C

C và D đều có miền gồm hai giá trị. Theo thứ tự biến, chọn C trước và thử giá trị nhỏ hơn trước.

Thử C = 3:

- Ràng buộc D < C yêu cầu D < 3.
- Nhưng D ≥ 3 nên không có giá trị D hợp lệ.

Phát hiện dead end, quay lui và hủy phép gán C = 3.

Thử C = 4:

- D < 4 và D ≥ 3 ⇒ D = 3.

Gán C = 4, D = 3.

Bước 4: Gán E

Do D = 3 và D ≠ E, miền E còn 4.

Chọn giá trị nhỏ nhất trước:

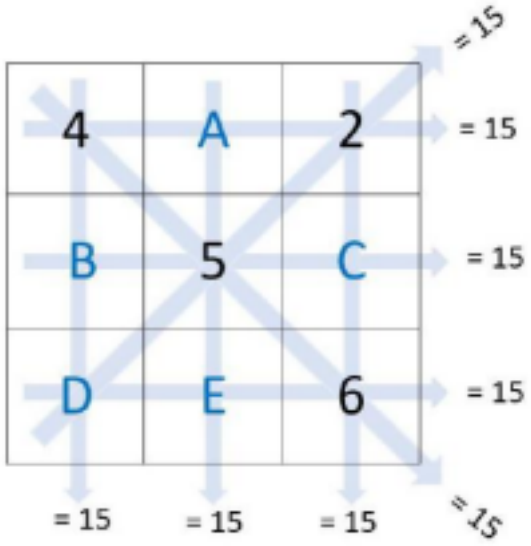
E = 1.

Máy bay	**Khung giờ**	**Đường băng**
A	2	Quốc tế
B	1	Quốc tế

C	4	Quốc tế
D	3	Nội địa
E	1	Nội địa

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

1. Biến

Các ô chưa biết là A, B, C, D, E. Các ô đã biết là 4, 2, 5, 6.

$$X = \{A, B, C, D, E\}$$

2. Miền giá trị

Các số từ 1 đến 9 phải xuất hiện đúng một lần. Do các số 2, 4, 5, 6 đã được sử dụng, miền ban đầu của các biến là:

$$D_A = D_B = D_C = D_D = D_E = \{1, 3, 7, 8, 9\}$$

3. Các ràng buộc

Tổng mỗi hàng, mỗi cột và hai đường chéo chính bằng 15.

- Hàng 1: $4 + A + 2 = 15$
- Hàng 2: $B + 5 + C = 15$
- Hàng 3: $D + E + 6 = 15$
- Cột 1: $4 + B + D = 15$
- Cột 2: $A + 5 + E = 15$
- Cột 3: $2 + C + 6 = 15$
- Đường chéo chính: $4 + 5 + 6 = 15$
- Đường chéo phụ: $2 + 5 + D = 15$

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Bỏ qua ràng buộc `AllDifferent` như đề cho phép, ta có các ràng buộc giữa từng cặp biến:

- $B + C = 10$ (hàng 2).
- $D + E = 9$ (hàng 3).
- $B + D = 11$ (cột 1).
- $A + E = 10$ (cột 2).

Đồ thị ràng buộc:

- Các đỉnh: A, B, C, D, E.
- Các cạnh: $B - C$, $D - E$, $B - D$, $A - E$.

Các ràng buộc hàng 1, cột 3 và đường chéo phụ chỉ liên quan đến một biến chưa biết nên được xử lý như ràng buộc đơn:

- $A = 9$
- $C = 7$
- $D = 8$

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Bước 1: Áp dụng ràng buộc đơn

Hàng 1:

$$4 + A + 2 = 15 \Rightarrow A = 9$$

Cột 3:

$$2 + C + 6 = 15 \Rightarrow C = 7$$

Đường chéo phụ:

$$2 + 5 + D = 15 \Rightarrow D = 8$$

Bước 2: Lan truyền ràng buộc

Từ cột 1:

$$4 + B + D = 15$$

Thay $D = 8$:

$$4 + B + 8 = 15 \Rightarrow B = 3$$

Từ cột 2:

$$A + 5 + E = 15$$

Thay $A = 9$:

$$9 + 5 + E = 15 \Rightarrow E = 1$$

Bước 3: Kiểm tra nghiệm

Ta thu được:

A=9,B=3,C=7, D=8,E=1

Magic Square hoàn chỉnh:

4	9	2
3	5	7
8	1	6

Kiểm tra tổng:

- Hàng: $4 + 9 + 2 = 15;\ 3 + 5 + 7 = 15;\ 8 + 1 + 6 = 15$.
- Cột: $4 + 3 + 8 = 15;\ 9 + 5 + 1 = 15;\ 2 + 7 + 6 = 15$.
- Đường chéo: $4 + 5 + 6 = 15;\ 2 + 5 + 8 = 15$.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Bước 1: Chọn biến theo MRV

- AJ có 3 giá trị.
- Sunny có 3 giá trị.
- Ellyse có 2 giá trị.

Theo MRV, chọn Ellyse trước.

LCV chọn màu ít hạn chế miền của hai bạn còn lại nhất. Đỏ xuất hiện trong miền của cả AJ và Sunny, còn Hồng chỉ xuất hiện trong miền của Sunny. Vì vậy, chọn Hồng.

Gán Ellyse = Hồng.

Bước 2: Chọn biến tiếp theo

Sau khi Ellyse chọn Hồng:

- AJ =
- Sunny =

Theo MRV, chọn Sunny trước.

Sunny chọn Đỏ theo thứ tự phá hòa.

Gán Sunny = Đỏ.

Bước 3: Gán AJ

AJ không thể chọn Đỏ vì Sunny đã chọn Đỏ.

Miền còn lại của AJ là . Chọn Vàng theo thứ tự ưu tiên.

Gán AJ = Vàng.

Nghiệm cuối cùng:

- Ellyse = Hồng
- Sunny = Đỏ
- AJ = Vàng

Số lần backtrack: 0. Không có phép gán nào gây mâu thuẫn, nên không cần quay lui.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Maximum Remaining Values (chọn biến có nhiều giá trị nhất) sẽ ưu tiên AJ hoặc Sunny vì mỗi bạn có 3 giá trị. Nếu hòa, chọn AJ trước theo thứ tự tên.

Most Constraining Value (MCV) chọn giá trị gây hạn chế nhiều nhất cho các biến khác.

Bước 1: Chọn AJ trước. Các màu Đỏ, Vàng, Xanh lá đều có thể hạn chế miền của Sunny hoặc Ellyse. Theo thứ tự màu, chọn Đỏ.

AJ = Đỏ.

Bước 2: Sunny và Ellyse còn miền:

- Sunny =
- Ellyse =

Chọn Sunny vì theo thứ tự biến. Theo MCV, chọn Vàng vì màu này chỉ còn trong miền của Sunny, không ảnh hưởng đến Ellyse.

Sunny = Vàng.

Bước 3: Ellyse chọn Hồng.

Ellyse = Hồng.

Nghiệm cuối cùng:

- AJ = Đỏ

- Sunny = Vàng
- Ellyse = Hồng

Số lần backtrack: 0.

So sánh: Cả hai cách đều tìm được nghiệm mà không cần quay lui. MRV thường giúp phát hiện mâu thuẫn sớm bằng cách chọn biến ít giá trị nhất, trong khi Maximum Remaining Values ưu tiên biến có nhiều lựa chọn nhất. LCV ưu tiên giá trị ít hạn chế các biến khác, còn Most Constraining Value ưu tiên giá trị gây nhiều hạn chế hơn.

--- Hết ---